

PocketTrainer

Second Follow-up — Verifying Fixes + Firewall / Encryption / SIEM Deep-Dive

Prepared using: NIST Cybersecurity Framework (CSF) 2.0

REPORT DATE September 18, 2026

BASELINE v1 (first review) and v2 (first follow-up)
reports — see Appendix for both

METHOD Source diff + Git history re-analysis of the
newly supplied archive vs. both prior reviews

CLASSIFICATION CONFIDENTIAL — security-sensitive findings

0

CRITICAL

0

HIGH

2

MEDIUM

0

LOW

Table of Contents

- 1. Executive Summary
- 2. Scope & Methodology
- 3. Remediation Verification (Prior Findings)
- 4. Open Findings Register (This Review)
- 5. Detailed Findings
- 6. Firewall & Network Security
- 7. Encryption (In Transit & At Rest)
- 8. SIEM & Security Monitoring
- 9. Other Attack-Surface Review
- 10. NIST CSF 2.0 Function Alignment
- 11. Prioritized Remediation Roadmap
- 12. Recommendations Quick-Reference
- 13. Positive Observations
- 14. Appendix — Methodology & Files Reviewed

1. Executive Summary

This is the second follow-up review of PocketTrainer. It compares a newly supplied copy of the project against both prior reports (the original audit and the first follow-up) and re-examines Git history across all three archives. The request for this pass was to confirm the remaining open items are actually resolved, take the developer's word that the leaked credentials have been rotated rather than treat that as still open, and give a bottom-line verdict on production readiness across the full attack surface covered so far — firewall/network, encryption, SIEM/monitoring, and other attack vectors.

Result: this was a very clean remediation pass. Of the nine items left open after the first follow-up, **six are now fully fixed and verified directly from source:** JWT session revocation (F13), the chat endpoint's rate limiter being wired in (F14), the workout controller's error handling (F15), the OTP-cooldown/weight-history schema fields (F16), the unnecessary CORS credentials flag (F19), and the unused resend dependency (F20). A dedicated commit — "fix(security): resolve F13-F16, F19, F20 (token revocation, chat limiter, error handling, schema persistence)" — addresses five of these in one pass, cleanly matching this report's own finding IDs.

F12 (credential rotation) is treated as resolved, per your confirmation that the MongoDB, JWT, Gmail, and RapidAPI credentials were rotated in Vercel. This is marked REPORTED FIXED rather than FIXED for consistency with how this report labels anything not independently re-derived from source — but unlike the last pass, there's now supporting evidence either way: the newly supplied archive contains **no backend/.env file at all** (previously it always shipped with live values), which is consistent with secrets now living only in Vercel and not in a local file that could leak again.

Two items remain genuinely open, and both are process/observability gaps rather than exploitable vulnerabilities: SIEM/monitoring is now partially addressed (Sentry APM error tracking was added — see F17) but there's still no structured request logging or authentication-anomaly alerting; and there is still no written incident-response/secret-rotation runbook (F18). One informational hygiene item persists (F21 — the delivered archive still bundles .git history and node_modules, though it no longer bundles the .env file). **No CRITICAL, HIGH, or unresolved application-layer vulnerability remains open.**

Bottom line on production readiness: from an application-security standpoint — authentication, session handling, input validation, injection/XSS/CSRF/SSRF/IDOR, rate limiting, and error handling — PocketTrainer is in good shape for production. It is *not yet* "perfectly" secure in the full sense of the request, for two reasons that are about process rather than code: (1) there's no monitoring/alerting layer to detect an attack in progress, and (2) two infrastructure-level settings that live outside source control — the MongoDB Atlas network access list and Vercel's Firewall configuration — still can't be verified from a code review and should be checked directly (Section 6). Close those two loops and this report has nothing outstanding above informational severity.

2. Scope & Methodology

In scope:

- Everything in scope for the prior reviews (mobile client, Express/MongoDB backend, third-party integrations, secrets handling) — see the earlier reports for the full list.
- Changed files in this pass: backend/models/User.js (tokenVersion, lastOtpSentAt, weightHistory fields), backend/middleware/authMiddleware.js (session revocation check), backend/routes/chatRoutes.js (chatLimiter wiring), backend/server.js (Sentry APM, CORS cleanup), backend/package.json (dependency changes).
- A dedicated deep-dive into firewall/network posture, encryption in transit and at rest, SIEM/monitoring capability, and a broader attack-surface pass (injection, XSS, CSRF, DoS, SSRF, supply-chain) — carried forward and updated from the first follow-up.

Method: all three archives (original, first follow-up, and this one) were extracted independently, source files were diffed pairwise to identify what changed at each step, and every copy's Git history was replayed (`git log --all`, `git show`) to confirm whether specific commits — the secret's original addition, its deletion, each fix commit, and the absence of any new `.env` commit — behaved as claimed. As before, this is a static/manual source review; no dynamic testing was performed against the live Vercel deployment, MongoDB Atlas cluster, or GitHub repository, and no infrastructure-level settings (Atlas Network Access rules, Vercel Firewall configuration) were directly inspected since they are not stored in source control.

Out of scope (unchanged from prior report): mobile OS-level hardening (root/jailbreak detection), the internals of the third-party Modal AI model, and any settings that live only in the Vercel or MongoDB Atlas dashboards.

3. Remediation Verification (Prior Findings)

Status of every finding raised across both prior reports, based on a direct diff of the source and a replay of all three archives' Git history.

#	Prior finding	Status	Evidence
F1 / F12	Live secrets committed to Git history / plaintext .env; credentials never rotated	REPORTED FIXED	History was purged in the first follow-up. Credentials have now been rotated in Vercel per the developer's confirmation. The newly supplied archive contains no backend/.env file at all, consistent with secrets no longer living in a local file.
F2	No rate limiting/lockout on auth endpoints	FIXED	backend/middleware/rateLimiter.js (authLimiter, otpLimiter, chatLimiter) is wired into /login, /register, /forgot-password, /reset-password, and — as of this pass — the chat route too (see F14, now also fixed).
F3 / F13	Weak JWT secret fallback + long-lived, non-revocable tokens	FIXED	generateToken() throws if JWT_SECRET is unset, tokens carry a tokenVersion claim, and authMiddleware.protect() now rejects a token whose tokenVersion doesn't match the current user record. Password reset and password update both increment tokenVersion, invalidating all previously issued sessions.
F4 / F19	Overly permissive CORS (all origins allowed / unneeded credentials:true)	FIXED	server.js uses an explicit allowedOrigins allowlist, and credentials: true has been removed now that it's confirmed unnecessary for the app's Bearer-token auth model.
F5	Auth token stored unencrypted in AsyncStorage	FIXED	services/tokenService.js uses expo-secure-store (Keychain/Keystore) on native platforms, unchanged and confirmed still in place.
F6	No password strength enforcement	FIXED	zod passwordSchema (8+ chars, 1 uppercase, 1 digit) remains applied to register, reset, and update-password flows.
F7	Unvalidated/unbounded input forwarded into AI prompts	FIXED	500-character message cap and bounded/paginated chat history remain in place, confirmed unchanged and working as intended.
F8 / F15	Raw internal error messages returned to clients	FIXED	workoutController.js's four handlers now call next(error) instead of building their own response, matching authController.js/chatController.js — the centralized error handler is used consistently across the whole API.
F9 / F17	No structured logging, monitoring, or alerting	PARTIALLY FIXED	Sentry (@sentry/node) APM was integrated in server.js, capturing unhandled exceptions with a global error handler. Structured request logging, auth-anomaly alerting, and log aggregation are still not present — see F17 in Section 4.
F10 / F18	No documented incident-response / secret-rotation runbook	NOT FIXED	Still no runbook file found. Three consecutive fix cycles following each report is a good informal track record, but nothing is written down yet.
F11 / F21	Full Git history, node_modules, and logs shipped in archive	PARTIALLY FIXED	The archive still includes the full .git directory and node_modules, but critically no longer includes backend/.env — the most sensitive file that used to ship with it is now absent.

#	Prior finding	Status	Evidence
F14	chatLimiter implemented but never applied to the chat route	FIXED	chatRoutes.js now imports and applies chatLimiter to both the GET and POST chat endpoints.
F16	Schema/persistence mismatch silently defeats the OTP resend cooldown	FIXED	User.js now declares lastOtpSentAt and weightHistory, explicitly commented '// F16: Declared paths so Mongoose strict mode persists them to MongoDB' — the cooldown and weight-trend tracking now actually persist.
F20	Unused 'resend' dependency adds supply-chain surface	FIXED	resend has been removed from backend/package.json; nodemailer remains as the sole email dependency.

Scorecard: 11 fixed or reported fixed, 2 partially fixed, 1 not fixed, out of 14 total findings raised across all reviews. Only the SIEM/monitoring depth (F17) and the incident-response runbook (F18) remain genuinely open, plus one informational packaging note (F21).

4. Open Findings Register (This Review)

Only three items remain: two medium-severity process/observability gaps and one informational packaging note. **No CRITICAL, HIGH, or open application-layer vulnerability remains.**

#	Finding	Severity	Primary NIST CSF 2.0 Mapping
F17	Sentry APM added, but still no structured logging or auth-anomaly alerting	MEDIUM	DE.CM-01, DE.AE-02
F18	No documented incident-response / secret-rotation runbook	MEDIUM	RS.MA-01, RC.RP-01
F21	Git history and node_modules still shipped in archive (no longer includes .env)	INFO	GV.SC-05, ID.AM-01

5. Detailed Findings

Six items from the open register (F12–F16, F19, F20) are now closed — see the verification table in Section 3 for exact evidence on each. The three write-ups below cover what's still open.

F17. Sentry APM added, but still no structured logging or auth-anomaly alerting

MEDIUM

NIST CSF 2.0 mapping: DETECT — DE.CM-01 (networks/systems monitored), DE.AE-02 (adverse events analyzed)

Impact: server.js now initializes Sentry (@sentry/node) when SENTRY_DSN is set, wraps the app with Sentry.setupExpressErrorHandler(), and explicitly calls Sentry.captureException(err) inside the centralized error handler. That's real, meaningful progress — unhandled exceptions and 500s are now captured with stack traces and context instead of vanishing into ephemeral Vercel function logs. What's still missing is the other half of Section 8's original recommendation: structured, correlatable request logging (who did what, when, from where) and alerting specifically tuned to security events — a spike in failed logins, repeated OTP failures for one account, or a burst of 429s from the rate limiters. Sentry will tell you an exception happened; it won't tell you someone is currently trying to brute-force a password.

Evidence:

```
backend/server.js – Sentry.init({ dsn: process.env.SENTRY_DSN, ... });  
Sentry.setupExpressErrorHandler(app); Sentry.captureException(err) inside the F8 error handler.
```

```
No structured logger (pino/winston) or log-aggregation destination was found;  
console.log/console.error calls in the controllers are unchanged.
```

Recommendations:

- Keep Sentry for exception/error tracking — it's doing its job. Add a structured logger (pino) on top for request-level events that aren't exceptions (successful logins, rate-limit rejections, OTP attempts).
- Configure a small number of Sentry alerts (or a lightweight custom check) specifically for security-relevant signals: repeated 401s from one IP/account, 429 bursts on auth routes, and OTP verification failures — Sentry's issue-alerting can cover some of this without a full separate SIEM.
- This is now closer to "mostly there" than "not started" — the remaining gap is narrower than in the last two reviews.

F18. No documented incident-response / secret-rotation runbook

MEDIUM

NIST CSF 2.0 mapping: RESPOND — RS.MA-01 (incident response plan executed); RECOVER — RC.RP-01 (recovery plan executed)

Impact: Still no runbook file exists in the repository. That said, the track record across three reviews now shows a real, repeatable process: each report has been followed by a dedicated fix commit ("feat: implement security audit fixes...", "fix(security): resolve F13-F16, F19, F20...", "feat(monitors): integrate sentry...") that maps cleanly onto this report's own finding IDs, and the credential rotation that was missed after the first follow-up was completed once flagged again. The remaining gap is purely that this process lives in your head rather than in a document a teammate (or a future you, under pressure) could follow without re-deriving it.

Recommendations:

- Write a one-page runbook: who to notify, how to rotate each class of secret (DB, JWT, email, third-party API keys), how to force-invalidate sessions (the tokenVersion mechanism added for F13 makes this concrete now — document the exact steps), and where to check for signs of misuse.

- Explicitly include "rotate any secret that was ever committed, even if since removed from the working tree" as its own line item, since that was the one step that took two review cycles to complete.

F21. Git history and node_modules still shipped in archive (no longer includes .env)

INFO

NIST CSF 2.0 mapping: GOVERN — GV.SC-05 (supply chain / handling of sensitive artifacts); IDENTIFY — ID.AM-01 (asset inventory)

Impact: The delivered archive still includes the full .git directory and node_modules for both app and backend, plus both previous self-generated audit reports inside the repository itself. The meaningful change since the last review: **backend/.env is no longer present anywhere in the archive** — the single most sensitive artifact that used to ship with every delivered copy of this project is now absent, which is a genuine hygiene improvement and consistent with the F12 rotation story (secrets now live in Vercel, not in a local file). What's left is lower-stakes packaging noise rather than a live credential risk.

Recommendations:

- When sharing for review or handoff, export a clean copy (git archive from a fresh clone, or exclude .git/, node_modules/, and .expo/) unless full history is specifically needed. Not urgent given .env is already excluded.

6. Firewall & Network Security

PocketTrainer's backend runs as Vercel serverless functions in front of MongoDB Atlas — there is no self-managed server or on-prem firewall to configure. "Firewall" for this architecture means three separate controls: Vercel's edge-level protections, MongoDB Atlas's network access list, and the application's own request-level throttling. None of the first two are visible from source code, since they're configured in each provider's dashboard rather than in a committed file — so this section states what could and couldn't be verified, and what to check.

Control / Area	Status	Assessment
Vercel Firewall / DDoS mitigation	NEEDS VERIFICATION	Vercel provides basic DDoS protection by default and an optional Firewall/Attack Challenge Mode as a dashboard setting. No <code>vercel.json</code> <code>firewall/rules</code> configuration is present in the repo, so this is either using platform defaults or hasn't been reviewed — confirm in the Vercel project dashboard.
MongoDB Atlas Network Access (IP allowlist)	NEEDS VERIFICATION	Not visible from source. Serverless functions have dynamic egress IPs, which commonly leads teams to allowlist <code>0.0.0.0/0</code> ("allow from anywhere") in Atlas just to keep deployments working — that setting alone would mean the database's network-level firewall is effectively disabled. This should be explicitly checked.
Application-level rate limiting	FIXED	<code>express-rate-limit</code> now covers <code>auth/OTP</code> routes and the <code>chat/AI</code> route (<code>chatLimiter</code> wired in). This is a useful but not sufficient control on its own: it operates per-IP by default and can be diluted by an attacker using many source IPs, which is why an edge-level WAF is still worth layering on top (see recommendations).
Payload size limits	FIXED	<code>server.js</code> sets <code>express.json({ limit: '100kb' })</code> , which blocks oversized JSON bodies being used as a crude denial-of-service/resource-exhaustion vector.
Function execution timeout	FIXED	<code>vercel.json</code> caps <code>server.js</code> and <code>api/**/*.*js</code> at <code>maxDuration: 60</code> seconds, bounding worst-case compute cost per request (relevant given the paid AI backend call).

Recommendations:

- Confirm and, if not already enabled, turn on Vercel's Firewall / Attack Challenge Mode for the production deployment, and review Vercel's built-in rate-limiting/bot-protection rules as a defense-in-depth layer above `express-rate-limit`.
- Check the MongoDB Atlas Network Access list specifically for a `0.0.0.0/0` entry. If serverless egress IPs make a tight allowlist impractical, use MongoDB Atlas's own IP Access List combined with database-user scoping (least-privilege roles per credential) rather than leaving the network layer fully open.
- Consider fronting the API with a proper WAF (Vercel's own, or Cloudflare in front of the Vercel deployment) if the app's user base or attack exposure grows — application-level rate limiting alone won't stop a distributed attack.

7. Encryption (In Transit & At Rest)

Control / Area	Status	Assessment
Transport encryption — API traffic	FIXED	Vercel serves all deployments over HTTPS/TLS by default and the mobile client's <code>API_BASE_URL</code> in <code>axiosConfig.js</code> is an <code>https://</code> URL. Helmet is applied, providing baseline security headers.
Transport encryption — database connection	FIXED	The <code>MONGO_URI</code> uses the <code>mongodb+srv://</code> scheme, which for MongoDB Atlas negotiates TLS by default for the driver-to-cluster connection.
Encryption at rest — database	NEEDS VERIFICATION	MongoDB Atlas encrypts data at rest by default on its managed clusters, but this is a platform default to confirm in the Atlas project settings rather than something visible from the connection string alone.
Encryption at rest — application secrets	NOT FIXED	<code>backend/.env</code> stores all credentials in plaintext on disk with no encryption or secrets-manager wrapper (this is F12/F1's root cause — see Section 5).
Sensitive data in JWT payload	FIXED (already good)	<code>generateToken()</code> signs only <code>{ id }</code> , no email, password, or health data in the token payload — JWTs are base64-encoded and readable by anyone who intercepts them, so keeping the payload minimal is the right call and was already the case in both reviews.
Field-level encryption for sensitive health fields	NOT FIXED	Date of birth, weight, and weight history are stored as plain fields in MongoDB with no additional field-level encryption. Relies entirely on Atlas's at-rest encryption and standard access control.
Certificate/transport pinning (mobile client)	NOT FIXED	No certificate pinning was found in the Axios configuration — the app trusts the device's standard certificate store, which is normal for most apps but offers less protection against a compromised CA or a MITM proxy on a compromised device.

Recommendations:

- Confirm encryption-at-rest is enabled on the MongoDB Atlas cluster (default on all current Atlas tiers, but worth a one-time confirmation in project settings).
- This is the same underlying problem as F12: move secrets out of a plaintext `.env` and into an encrypted store (Vercel encrypted environment variables at minimum; a dedicated secrets manager for anything beyond a small project).
- For a health/fitness app, consider whether particularly sensitive fields (date of birth) warrant application-level (field) encryption in addition to Atlas's at-rest encryption, as defense-in-depth against a database-level compromise.
- Certificate pinning is optional for most consumer apps and adds maintenance overhead (pin rotation); it's worth considering only if the threat model includes sophisticated on-device attackers, which doesn't appear to be the current priority relative to the other findings in this report.

8. SIEM & Security Monitoring

There is currently no SIEM (Security Information and Event Management) capability of any kind. This section lays out what a minimally viable version would look like for an app at PocketTrainer's stage, since "add a SIEM" isn't itself an actionable step without more specificity.

Control / Area	Status	Assessment
Structured, centralized logging	NOT FIXED	All logging is console.log/console.error scattered across controllers (e.g. [GENERATION START], [AI WORKOUT SYNC], [SERVER ERROR]), with no request IDs, no consistent structure, and no aggregation — Vercel's function logs are short-retention and not designed as a log warehouse.
Authentication anomaly detection	NOT FIXED	No alerting exists for spikes in failed logins, 429 (rate-limited) responses, or repeated OTP failures — exactly the signal that would show F2's rate limiter is under active attack.
Database/query anomaly detection	NOT FIXED	No monitoring on MongoDB Atlas query patterns, connection counts, or unusual data access volume (Atlas itself offers this as a paid feature, but it doesn't appear to be configured/reviewed).
Error/exception tracking (APM)	NOT FIXED	No Sentry/Bugsnag/equivalent integration — errors are only visible by manually reading serverless function logs after the fact.
Audit trail for sensitive actions	NOT FIXED	No audit log for security-relevant events (password changes, failed auth attempts, admin-style actions) separate from ordinary application logs.

Recommendations (roughly in order of effort vs. value):

- **Start with structured logging.** Replace ad hoc console.log calls with a structured logger (pino is a good fit for serverless — low overhead, JSON output) that includes a request ID, user ID (when authenticated), route, and outcome on every request.
- **Ship logs somewhere persistent.** Forward structured logs to a log-management platform with real retention and search — options at this scale include Better Stack (Logtail), Axiom, Datadog, or the ELK/OpenSearch stack if self-hosting is preferred. This is the foundation a SIEM sits on top of.
- **Add an APM/error tracker** (Sentry is the common choice for Node/React Native) so exceptions are captured with stack traces and context automatically, instead of relying on someone noticing a 500 response.
- **Define 3–5 concrete alerts** before reaching for a full SIEM product: a spike in 401/429 responses on auth routes, repeated OTP verification failures for the same account, MongoDB connection errors, and any 500 rate above a small baseline. Route these to email/Slack at minimum.
- **Graduate to a SIEM proper once there's enough log volume/team size to justify it** (e.g. Microsoft Sentinel, Elastic Security, or a lighter SaaS SIEM) — for a project this size, the structured-logging + alerting steps above deliver most of the practical benefit at a fraction of the operational cost.

9. Other Attack-Surface Review

A pass across the classic web/mobile attack categories not already covered as standalone findings above.

Vector	Assessment	Status
NoSQL / query injection	All Mongoose queries observed use validated, typed values (email/password/etc. are parsed through zod schemas as plain strings before reaching <code>User.findOne()</code> , which prevents an attacker from injecting a MongoDB query operator like <code>\$gt</code> via a JSON body). No raw/unsanitized user input was found being interpolated into a query object.	FIXED (already good)
Cross-site scripting (XSS)	The client is a React Native app, not a web view rendering raw HTML — user- and AI-generated text (chat messages, workout notes) is rendered through native Text components rather than an HTML sink, which is the main XSS vector in web apps. Risk is low in the current mobile-only client; revisit if a web/admin dashboard is ever added.	OUT OF SCOPE
Cross-site request forgery (CSRF)	Authentication is entirely Bearer-token-based (Authorization header), not cookie/session-based — CSRF specifically exploits ambient cookie auth, so the current design is not exposed to it. The <code>CORS credentials:true</code> flag that was an unnecessary leftover here has since been removed.	FIXED (already good)
Denial of Service (DoS) — request volume	Auth/OTP routes and the chat/AI route are now both rate-limited.	FIXED
Denial of Service (DoS) — payload/resource exhaustion	JSON body size capped at 100kb, function execution capped at 60s, and chat messages capped at 500 characters — reasonable bounds against oversized-payload and long-running-request abuse.	FIXED (already good)
Server-Side Request Forgery (SSRF)	Outbound calls from the backend (to the Modal AI endpoint and the RapidAPI ExerciseDB) use fixed, environment-configured URLs/hosts — exercise names are passed as query values, not as URLs or hosts, so user input cannot redirect these requests to an attacker-controlled destination.	FIXED (already good)
Broken Object-Level Authorization (IDOR)	Workout, dashboard, and chat endpoints continue to scope data lookups to <code>req.user</code> (populated from the verified JWT) rather than trusting a client-supplied <code>user/record ID</code> — consistent with the previous review's finding.	FIXED (already good)
Supply-chain / dependency risk	No automated dependency scanning (npm audit / Dependabot / Snyk) is configured in CI. The unused <code>resend</code> dependency has since been removed. <code>package-lock.json</code> is committed, which is good practice for reproducible installs.	NOT FIXED
Secrets in logs	<code>console.log</code> statements reviewed print IDs, plan titles, and status text — no instance of a password, token, or API key being logged was found.	FIXED (already good)

Recommendations:

- Add automated dependency scanning to CI (Dependabot is free and simplest on GitHub; Snyk/npm audit are alternatives) so known-vulnerable packages are flagged automatically rather than relying on manual review.
- If a web-based admin dashboard or companion web app is ever built, revisit XSS specifically — the "out of scope" status here is a property of the current mobile-only client, not a guarantee that will still hold with a different frontend.

10. NIST CSF 2.0 Function Alignment

Updated from the prior review to reflect verified fixes. Protect has improved the most (rate limiting, CORS, secure storage, input validation all landed); Detect and Respond remain the weakest functions, consistent with F17/F18 and the SIEM assessment in Section 8.

Function	Change since last review	Posture	Key open items
GOVERN (GV)	No change — still no written secrets policy or data-classification standard, though the fix-cycle track record is now three reports strong.	Weak	F18, F21
IDENTIFY (ID)	Slightly improved — the unused dependency was removed, tightening the supply-chain footprint. Still no formal secrets/asset inventory.	Partial	F21
PROTECT (PR)	Now the strongest function: rate limiting (incl. chat), CORS allowlist without unneeded credentials, secure token storage, password policy, bounded AI input, JWT session revocation, consistent error handling, and rotated credentials are all in place.	Good	None open
DETECT (DE)	Improved — Sentry APM now captures exceptions. Still no structured request logging or security-specific alerting.	Partial (was Weak)	F17
RESPOND (RS)	Improved in practice across three review cycles (each report was followed by real fix commits, including the once-missed rotation) but still undocumented.	Weak (improving)	F18
RECOVER (RC)	No change — no application-level recovery/rotation plan documented, though the tokenVersion mechanism added for F13 gives a concrete tool a future runbook could reference.	Not evidenced	F18

11. Prioritized Remediation Roadmap

Immediate

0 – 7 days

- Confirm the F12 credential rotation at each provider directly (Atlas, Google, RapidAPI show the change), not only in Vercel's dashboard, and check provider access/billing logs for the exposure window.
- Manually verify the two settings this review can't see from source: MongoDB Atlas Network Access (watch for a 0.0.0.0/0 allowlist entry) and Vercel Firewall/Attack Challenge Mode status (Section 6).

Short-term

2 – 4 weeks

- Add structured request logging (pino) alongside the new Sentry APM integration, and define 3–5 concrete security alerts (auth failure spikes, repeated OTP failures, 429 bursts) (F17, Section 8).
- Write a one-page incident-response / secret-rotation runbook — the tokenVersion mechanism now in place gives it something concrete to reference for session invalidation (F18).
- Add automated dependency scanning (Dependabot/Snyk) to CI so future supply-chain issues surface automatically (Section 9).

Ongoing / Longer-term

1 – 3 months

- Move all secrets to a dedicated secrets manager beyond Vercel's built-in env vars, for defense-in-depth (Section 7).
- Graduate from Sentry-only error tracking to a full log-management/SIEM setup once log volume and team size justify it (Section 8).
- Consider a WAF (Vercel Firewall or Cloudflare) in front of the API as the app's user base grows (Section 6).
- Continue the pattern that's worked well so far: re-run a verification pass after each round of fixes.

12. Recommendations Quick-Reference

Only three items remain open — full context for each is in Section 4/5, 6, or 8 as noted.

#	Recommendation	Severity
F17	Add structured request logging on top of the new Sentry APM integration, plus a few concrete security alerts.	MEDIUM
F18	Write a short incident-response / secret-rotation runbook, referencing the new tokenVersion revocation mechanism.	MEDIUM
F21	Share a clean export (no .git/node_modules) when handing off or reviewing the project — not urgent now that .env is already excluded.	INFO

13. Positive Observations

Everything confirmed fixed across all three review cycles:

- The leaked MongoDB, JWT, Gmail, and RapidAPI credentials have been rotated in Vercel's environment variables, per the developer, and the delivered archive no longer bundles a backend/.env file at all (F12, reported fixed).
- JWT sessions can now be revoked server-side: a tokenVersion claim is checked on every request and incremented on password reset/update, immediately invalidating old tokens across all devices (F13, fixed).
- Rate limiting now covers login, register, OTP request/verification, and the chat/AI endpoint (F2 and F14, fixed).
- The mobile client stores its auth token in platform secure storage (Keychain/Keystore via expo-secure-store) instead of AsyncStorage (F5, fixed).
- Request bodies are validated with zod, including a real password-complexity policy (F6, fixed) — and this same validation is a big part of why NoSQL injection isn't practically exploitable here (Section 9).
- CORS is restricted to an explicit origin allowlist with the unnecessary credentials:true flag removed (F4/F19, fixed).
- Chat messages are length-capped and chat history is paginated/bounded (F7, fixed).
- The centralized, production-safe error handler is now used consistently across every controller, including workoutController.js (F8/F15, fixed).
- The User schema now declares lastOtpSentAt and weightHistory, so the OTP resend cooldown and weight-trend tracking actually persist as intended (F16, fixed).
- Sentry APM was integrated for exception tracking and error capture (part of F17, meaningfully improved).
- The unused resend dependency was removed, tightening the supply-chain footprint (F20, fixed).
- The JWT fallback secret was removed entirely; the app fails fast if JWT_SECRET is missing (part of F3, fixed).
- Passwords are hashed with bcrypt, Helmet is applied globally, the forgot-password flow avoids account enumeration, and reset codes are hashed before storage — unchanged and still good practice.
- Authentication is Bearer-token-based rather than cookie-based, which structurally avoids CSRF exposure (Section 9).
- JWT payloads contain only a user ID and tokenVersion — no email, password, or health data — limiting what's exposed if a token is intercepted.
- Endpoints continue to scope all data access to the authenticated req.user rather than trusting client-supplied IDs, avoiding IDOR (Section 9).
- Request payload size and function execution time are both capped, and outbound third-party calls use fixed, non-user-controlled URLs, closing off straightforward SSRF and basic resource-exhaustion vectors (Sections 6 and 9).

14. Appendix — Methodology & Files Reviewed

Comparison basis:

- Original archive: 1788620030803_PocketTrainer.rar — first audit (PocketTrainer_Security_Audit_Report.pdf).
- First follow-up archive: 1789417742385_pocket-trainer.rar — reviewed in PocketTrainer_Security_Audit_Report_v2.pdf.
- Current archive: 1789738256794_pocket-trainer.rar, diffed file-by-file against the first follow-up's extracted source tree.
- All three archives' .git directories were extracted and replayed independently (git log --all, git show, git ls-tree) to verify the secret-rotation, history-purge, and fix-commit claims across the full history.

Files changed since the last review, confirmed via diff:

- backend/models/User.js — added lastOtpSentAt, weightHistory, tokenVersion fields (F16, F13).
- backend/middleware/authMiddleware.js — added tokenVersion check for session revocation (F13).
- backend/controllers/authController.js — generateToken() now signs tokenVersion; both password-change paths increment it (F13).
- backend/controllers/workoutController.js — all four handlers now call next(error) (F15).
- backend/routes/chatRoutes.js — chatLimiter wired into both routes (F14).
- backend/server.js — Sentry APM integration added; CORS credentials:true removed (F17, F19).
- backend/package.json — @sentry/node added; resend removed (F20).
- No backend/.env file present anywhere in this archive (supports the F12 rotation report).

Confirmed unchanged from the prior review (spot-checked, not flagged in diff):

- backend/routes/authRoutes.js, workoutRoutes.js; backend/models/WorkoutPlan.js, ChatMessage.js, DailyLog.js
- backend/services/aiService.js, emailService.js, exerciseService.js; backend/config/db.js
- api/axiosConfig.js, services/tokenService.js, context/AuthContext.jsx

This report reflects a manual source-code and Git-history review only. It does not constitute a penetration test, and does not directly verify current MongoDB Atlas network/IAM configuration or Vercel Firewall settings (both are called out as "needs verification" in Section 6 rather than assumed). The F12 credential rotation is recorded as reported by the developer; this review found supporting evidence (no .env file shipped in this archive) but did not independently confirm the new values against each provider's dashboard, which remains outside the scope of a source review.